

PORTAL

DEPLOYMENT & INTEGRATION GUIDE

v0.0.7 · June 2026

portal.gripe

This guide documents the full setup, deployment, and integration of Portal – a two-person WebRTC video app with Solana-based key access and server-side face recognition for identity verification.

1. ARCHITECTURE OVERVIEW

Portal consists of three services:

Service	Stack	Role
Node.js server	Express + Socket.io	WebRTC signalling, key validation, face proxy
Face service	Python FastAPI + dlib	Face enrollment and verification
Solana program	Anchor 1.0.2 (Rust)	On-chain key purchase and validation

User flow:

1. User visits `buy.html`, connects Phantom wallet, purchases a key on-chain.
2. After payment confirms, webcam opens and their face is enrolled via the face service.
3. User shares the 8-character key with one other person.
4. Both open the portal, enter the key, and their faces are verified before the room is joined.
5. If a face doesn't match the enrolled one, both users see a warning – connection still proceeds.

2. TOOLCHAIN SETUP

2.1 Prerequisites

Install the following tools in order. Run all commands in Terminal (not PyCharm or any IDE).

2.2 Rust

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
. "$HOME/.cargo/env"
```

2.3 Solana CLI (Agave)

```
sh -c "$(curl -sSfL https://release.anza.xyz/stable/install)"
export PATH="$HOME/.local/share/solana/install/active_release/bin:$PATH"
```

■ Use `release.anza.xyz` – the old `release.solana.com` URL is dead.

2.4 Anchor 1.0.2

```
cargo install --git https://github.com/coral-xyz/anchor avm --locked
avm install 1.0.2
avm use 1.0.2
```

Verify: `anchor --version` should print `anchor-cli 1.0.2`

■ Do not use Anchor 0.29.x or 0.30.x – they pull in Solana 1.17/1.18 platform tools which cannot parse Cargo.lock v4 generated by Rust 1.78+.

3. SOLANA SMART CONTRACT

3.1 Project structure

```
portal/
├── programs/portal/
│   ├── src/lib.rs          ← Rust smart contract
│   ├── Cargo.toml
│   └── Anchor.toml
├── Cargo.toml              ← workspace root
├── target/
│   └── deploy/
│       ├── portal.so       ← compiled program
│       └── portal-keypair.json ← program keypair (keep safe)
```

3.2 Cargo.toml (workspace root)

```
[workspace]
resolver = "2"
members = ["programs/*"]

[profile.release]
overflow-checks = true
opt-level = "z"
lto = true
codegen-units = 1
```

3.3 programs/portal/Cargo.toml

```
[package]
name = "portal"
version = "0.1.0"
edition = "2021"

[lib]
crate-type = ["cdylib", "lib"]
name = "portal"

[features]
```

```

no-entrypoint = []
no-idl = []
no-log-ix-name = []
cpi = ["no-entrypoint"]
default = []
idl-build = ["anchor-lang/idl-build"]

[dependencies]
anchor-lang = { version = "1.0.2", features = ["init-if-needed"] }
solana-sha256-hasher = "3"

```

3.4 Key API changes from 0.29 → 1.0.2

Change	Old (0.29)	New (1.0.2)
CpiContext::new first arg	AccountInfo	Pubkey (use system_program::ID)
Hash import	anchor_lang::solana_program::hash	solana_sha256_hasher::hash
init-if-needed	works by default	requires features=["init-if-needed"]
IDL generation	auto	requires idl-build feature

3.5 Generate program keypair

```

mkdir -p target/deploy
solana-keygen new --outfile target/deploy/portal-keypair.json --no-bip39-passphrase
solana-keygen pubkey target/deploy/portal-keypair.json # → YOUR_PROGRAM_ID

```

Replace REPLACE_WITH_YOUR_PROGRAM_ID in: programs/portal/src/lib.rs
(declare_id!), Anchor.toml, public/index.html, buy.html.

3.6 Build

```
anchor build
```

■ If you get 'lock file version 4' errors, ensure you are using Anchor 1.0.2 (not 0.29/0.30). These older versions install Solana 1.17/1.18 tools which cannot handle modern Cargo lock files.

3.7 Deploy

Deploying to mainnet costs ~1.06 SOL (one-time, rent-exempt storage for 149KB binary). This SOL is recoverable if you ever close the program.

```

# Mainnet
solana config set --url mainnet-beta
anchor program deploy -- --with-compute-unit-price 5000

# Devnet (free, for testing)
solana config set --url devnet

```

```
solana airdrop 2    # or use faucet.solana.com
anchor program deploy -- --with-compute-unit-price 5000
```

■ Anchor.toml must have cluster = "mainnet" or cluster = "devnet" to match your target – it overrides the Solana CLI config.

■ On congested devnet, the deploy may time out mid-upload. Keep retrying – each run resumes from where it stopped. Use a Helius RPC for reliability.

3.8 After deployment – update the IDL

Replace the IDL stub in portal-keys.js with the generated IDL:

```
cat target/idl/portal.json
# Copy the output and replace the `const IDL = { ... }` block in portal-keys.js
```

4. FACE RECOGNITION SERVICE

A Python FastAPI service handles face enrollment and verification using the `face_recognition` library (dlib-based). It runs as a separate service alongside the Node.js server.

4.1 File: `face_service/main.py`

Two endpoints:

POST `/enroll` – receives `portal_key` + `image_b64`, extracts face encoding, stores to disk.

POST `/verify` – receives `portal_key` + `image_b64`, compares against stored encoding. Returns `match (bool)`, `confidence (0-1)`, and `reason`.

Face encodings are stored as a JSON file (`face_data/encodings.json`) – one 128-float array per key.

4.2 File: `face_service/requirements.txt`

```
face_recognition==1.3.0
fastapi==0.111.0
uvicorn==0.30.1
pillow==10.3.0
numpy==1.26.4
pydantic==2.7.1
```

4.3 Verification logic

The `face_recognition` library computes a 128-dimensional face descriptor (same as dlib). Two faces are compared by Euclidean distance. Threshold: 0.5 (lower = stricter).

```
distance = face_recognition.face_distance([stored_encoding], live_encoding)[0]
confidence = 1 - distance    # 1.0 = perfect match
match = distance < 0.5
```

■ No enrollment (`reason: no_enrollment`) is treated as a pass – keys purchased before this feature existed work normally.

4.4 Deploying the face service on Render

Create a second Web Service on Render pointing at the `face_service/` subdirectory:

Setting	Value
---------	-------

Root directory	face_service
Build command	pip install -r requirements.txt
Start command	python main.py
Environment var	FACE_DATA_FILE=/data/encodings.json

■ dlib requires C++ compilation. Render's Python environment supports it but the first build takes 5-10 minutes. Add a persistent disk to Render to survive face_data/ across deploys.

5. NODE.JS SERVER

5.1 Face proxy endpoints added to server.js

The Node server proxies /face/enroll and /face/verify to the Python service. Set FACE_SERVICE_URL in the environment to point at your Render face service URL.

```
const FACE_SERVICE_URL = process.env.FACE_SERVICE_URL || "http://localhost:8000";

app.post("/face/enroll", (req, res) => faceProxy("/enroll", req.body, res));
app.post("/face/verify", (req, res) => faceProxy("/verify", req.body, res));
```

5.2 face_warning socket event

When a user's face fails verification, the client emits a face_warning event. The server relays it to the other user in the room.

6. FRONTEND CHANGES

6.1 buy.html – face enrollment

After the key is revealed and copied, the page opens the webcam and prompts the user to scan their face. The captured frame is sent to /face/enroll. The user can skip enrollment – the key still works, just without face verification.

6.2 public/index.html – face verification on join

The join() function now runs face verification before emitting the socket join event:

1. Camera starts (already required for video call).

2. A frame is grabbed using ImageCapture API.
3. Frame is sent to /face/verify with the key code.
4. If match or no_enrollment → join proceeds silently.
5. If mismatch → socket._faceWarning is set. After the peer connects, both users see a warning in the status bar.

7. ENVIRONMENT VARIABLES

Create a `.env` file in the project root (never commit this file):

```
# TURN server (for reliable connections across networks)
TURN_URL_UDP=turn:YOUR_VPS_IP:3478?transport=udp
TURN_URL_TCP=turn:YOUR_VPS_IP:3478?transport=tcp
TURN_USERNAME=portal
TURN_CREDENTIAL=your_turn_password

# Solana
SOLANA_RPC_URL=https://api.mainnet-beta.solana.com
PROGRAM_ID=YOUR_PROGRAM_ID
TREASURY_PUBKEY=YOUR_WALLET_PUBKEY
SERVER_KEYPAIR=/etc/secrets/id.json

# Key system
PORTAL_KEYS_ENABLED=true

# Admin (generate gifted keys)
ADMIN_SECRET=pick_a_strong_random_secret
ADMIN_KEYS=OPTIONAL_COMMA_SEPARATED_GIFTED_KEYS

# Face service
FACE_SERVICE_URL=https://your-face-service.onrender.com
```

8. DEPLOYING TO RENDER

Two web services, both from the same GitHub repo:

8.1 Node.js server (portal)

```
Root directory:  . (project root)
Build command:   npm install
Start command:   node server.js
Secret file:      /etc/secrets/id.json ← your Solana keypair JSON
```

8.2 Face service (portal-face)

```
Root directory:  face_service
Build command:   pip install -r requirements.txt
Start command:   python main.py
Disk:            Mount a persistent disk at /data ← stores face encodings
```

8.3 Environment variables (Node service)

Add all variables from Section 7 under Environment in the Render dashboard. For SERVER_KEYPAIR, upload your id.json as a Secret File at path /etc/secrets/id.json.

8.4 Custom domain

Point portal.gripe to your Render service by adding a custom domain in Render → Settings → Custom Domains, then update your DNS CNAME record.

9. ADMIN & GIFTED KEYS

Generate a free key at runtime (lost on server restart):

```
curl -X POST https://portal.gripe/admin/generate-key \
  -H "x-admin-secret: YOUR_ADMIN_SECRET"
# → { "portalKey": "XXXXXXXX" }
```

For permanent gifted keys that survive restarts, add them to .env:

```
ADMIN_KEYS=KEY1XXXX,KEY2XXXX
```

10. KEYBOARD SHORTCUTS

Shortcut	Action
Space	Mute / unmute microphone
Cmd+D	Toggle camera on/off
Cmd+V	Minimize / restore video box
Cmd+C	Maximize / minimize chat
Cmd+R	Refresh / disconnect portal
Double-tap / dblclick video	Toggle remote control of peer's view

11. VERIFICATION CHECKLIST

Once everything is deployed, run through this checklist:

■ Node server responds at your Render URL (visit /ice-config – should return JSON)

- Face service responds at its Render URL (visit /health – should return {status: ok})
- buy.html loads and displays the key price
- Phantom wallet connects on buy.html
- Key purchase completes and face enrollment prompt appears
- Face scan succeeds and returns enrolled status
- On index.html, entering the key starts face verification
- Two browser tabs/devices can connect in the same room
- Video and audio stream between both peers
- Mismatched face shows warning on both sides
- Spacebar mutes audio, Cmd+D kills video
- Double-click video enables remote control